14/9/26 Practical 10.

AIM:- b) Design and configure an internetwork using wireless router, DHCP Server and internet cloud.

Addressing Table.

Device	Interface	IP Address	Subnet Mask	Default Gateway.
PC	Ethernet 0	DHCP		192.168.0.1
Wireless Router	LAN	192.168.0.1	255.255.255.0	
Wireless Router	internet	DHCP		
Cisco.com in Server	Ethernet 0	208.67.220.220	255.255.255.0.	
Laptop	wireless 0	DHCP		

Objectives:

Part 1: Build a Simple Network in the Logical Topology workspace.

Part 2: Configure the network Devices.

Part 3: Test Connectivity between network Devices.

Part 4: Save the File and close Packet Tracer.

Part 1: Build a Simple Network in the Logical Topology workspace.

Step 1: Launch Packet Tracer.

Part 2: Configure the Network Devices.

Step 1: Configure the wireless router.

a. Create the wireless router.

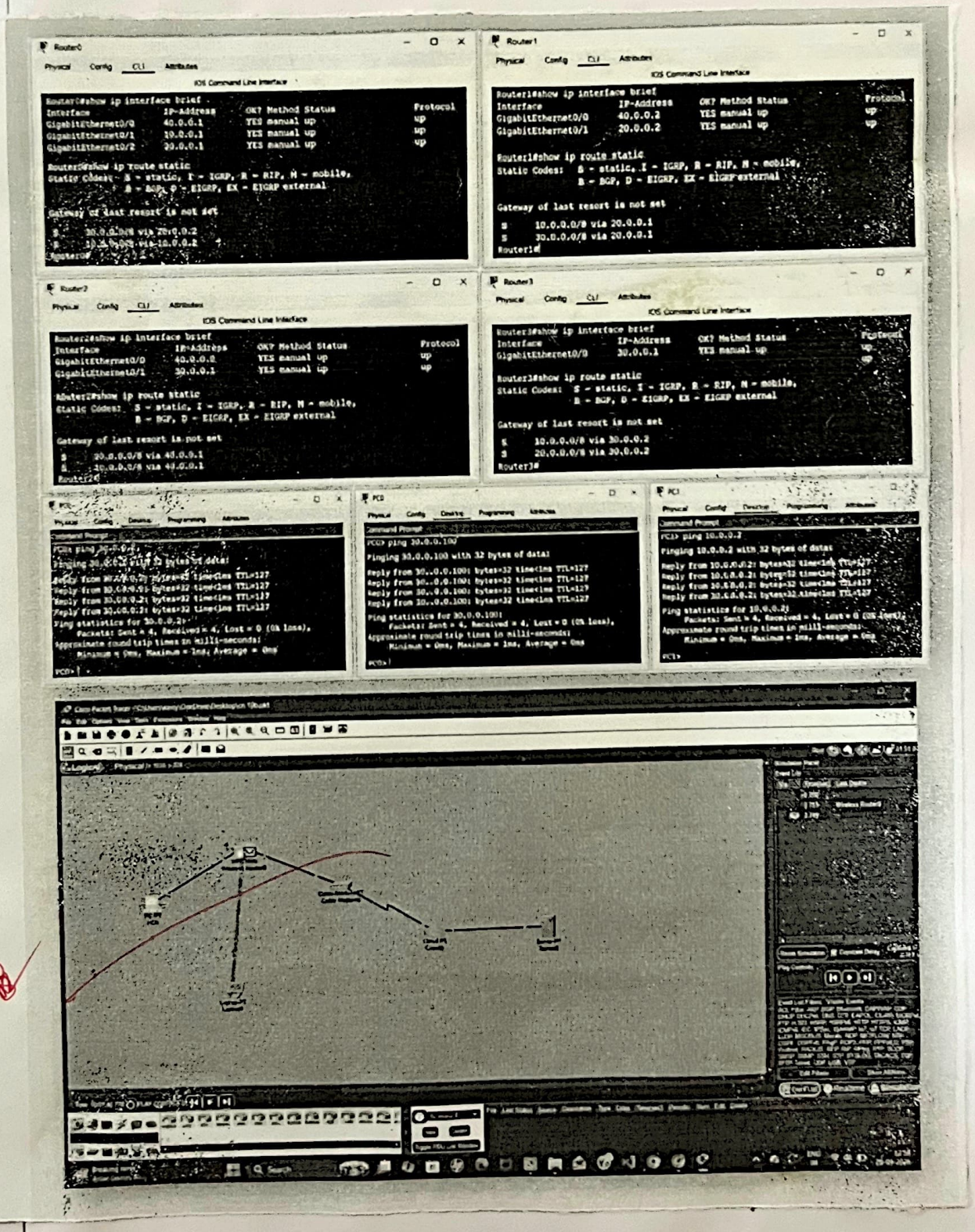
b. Click on the save settings tab.

Step 2 : Configure the laptop.

Step 3 : Configure the PC.

Step 4 : Configure the Internet Cloud.

Step 5 : Configure the Cisco.com server.



22/9/22 Practical 11.

AIM: a) Simulate static Routing configuration using Cisco Packet Tracer.

Static routes are the routes you manually add to the router's routing table. The process of adding static routes to the routing table is known as static routing. Let's take a packet tracer Example to understand how to use static routing to create and add a static route to the routing table.

Setting up a practise lab.

Create a packet tracer lab as shown in the following image or download the following precreated lab and load it on Packet Tracer.

Packet Tracer Lab with Initial IP Configuration.

In this lab, each network has two routes to reach. we will Configure one route as the main route and another route as the backup route. If the link bandwidth of all routes is the same, we use the route that has the least number of routers as the main route. If the link bandwidth and the number of routers are the same, we can use any route as the main route and another route as the backup route.

Router 1 requirements.

* Create two routes for network 10.0.0.0/8 and Configure the first route (via - Router 0) as the main route and the second route (via - Router 1) as a backup route.

Create two routes for network 40.0.0.0/8 and configure the first route (via_Router 0) as the main route and the second route (via Router-2) as a backup route.

* Verify the router adds only main routes.

Router 2 requirements.

Create static routes for network 10.0.0.0/8 and network 30.0.0.0/8 and verify the router adds both routes to the routing table.

Verifying Static routing.

On Router 0, we Configured two routes for network 30.0.0.0/8. These routes are via Router1 and via Router 2. We set the first route (via_Router 1) as the main route and the second route as the backup route. we can verify this Configuration in two ways.

By Sending Ping requests to a PC of network 30.0.0.0/8 the tracing the path they take to reach the network 30.0.0.0/8. For this you can use 'tracert) Command on a PC of network 10.0.0.0/8. The tracer command Sends ping request to the destination host and tracks the path they take to reach the destination.

By listing the routing table entries on Router 0. Since a router uses the routing table to forward data packets, you can check the routing table to figure out the route the router uses to forward data packets for each destination.

Practical 11.

AIM: b) Simulate RIP using Cisco Packet Tracer.

Assign IP address to PCs.

Double click PCs and click Desktop menu item and click IP Configuration. Assign IP address referring the above table.

Assign IP address to interfaces of routers.

Double click Router 0 and click CLI and press ENTER key to access the command prompt of Router 0.

we need to Configure IP address and other parameters on interfaces before we could actually use them for routing. Interface mode is used to assign IP address and other parameters. Interface mode an be accessed from global Configuration mode. Following Commands are used to access the global Configuration mode.

Router > enable.

Route # Configure terminal.

Enter configuration commands, one per line. End with CNTL/Z.

Router # show Controllers Serial 0/0/0

Interface Serial 0/0/0

Hardware is Power QUICC MPC 860

DCE V. 35, clock rate 2000000

[output omitted]

Router # Configure terminal.

Enter Configuration Commands, one per line. End with CNTL/Z

Router (Config)# interface Serial 0/0/0

Router (config-if) # IP address 192.168.1.249 255.255
255.252

Router (config-if) # ip clock rate 64000

Router (Config-if) # bandwidth 64.

Router (config-if) # no shutdown.

Router (config-if) # exit.

Router (config) # interface Serial 0/0/1.

Router (Config-if) # IP address 192.168.1.254. 255.255
255.252

Router (config-if) # IP address clock rate 64000

Router(Config-if) # clock bandwidth.

64 Router (Config-if) # no

Shutdown Router (config-

Router 1.

Router > enable.

Router # configure

terminal.

Enter configuration commands, one per line end with CNTL/Z

Router (Config) # interface serial 0/0/0.

Router (Config-if) # no shutdown.

Router (config-if) # exit.

Router (config) # interface Serial 0/0/1

Router (Config-if) # IP address 192.168.1.246. 255.255.255.252

Router (config-if) # clock rate 64000

Router (Config-if) # bandwidth
64 Router (Config-if) # no

Router 1.

Router 1 (Config) # router rip.

Router 1 (Config - router) # network 192.168.1.244.

Router 1 (Config - router) # network 192.168.1.248.

Router. 2.

Router 2 (config) # router rip.

Router 2 (config - router) # network 20.0.0.0.

Router 2 (Config - router) # network 192.168.1.252.

Router 2 (config - router) # network 192.168.1.244.

That's it. our network is ready to take the advantage of RIP routing. To verify the setup we will use ping Command. Ping Command is used to test the Connectivity between two devices. Access the Command prompt of PC1 and use ping Command to test the Connectivity from PC0.

RIP Protocol automatically manage all routes for us. If one route goes down, it automatically switches to another available. To explain this process more clearly we have added one more route in our network.

Currently there are two routes between PC0 and PC1.

Route 1.

PC0 / Source / destination — 10.0.0.2 / <==> Router 0

[Fast Ethernet 0/1 - 10.0.0.1] <==> Router 0

[Serial 0/0/1 - 192.168.1.254) <==> Router 2

[Serial 0/0/1 - 192.168.1.253) <==> Router 2

[Fast Ethernet 0/0 - 20.0.0.1] <==> PC1. (Destination/ Source. 20.0.0.2]

Route 2.

PC01 Source/ destination - 10.0.0.2/ <==> Router 0.

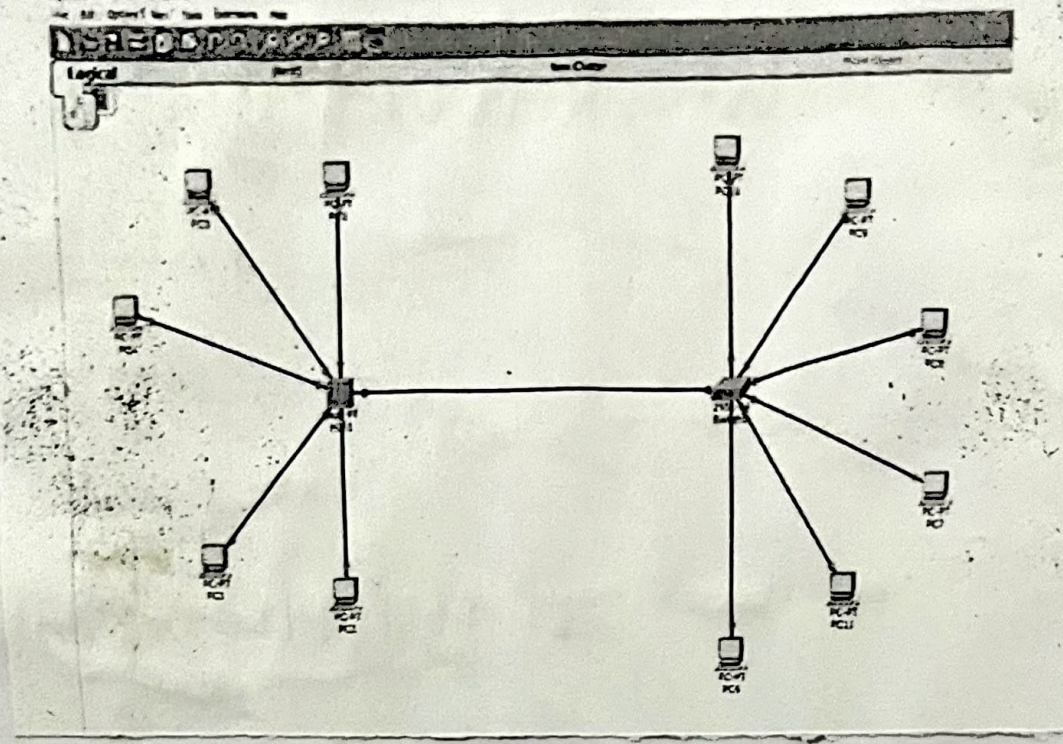
/ Fast Ethernet 0/1 - 10.0.0.1 <==> Router 0

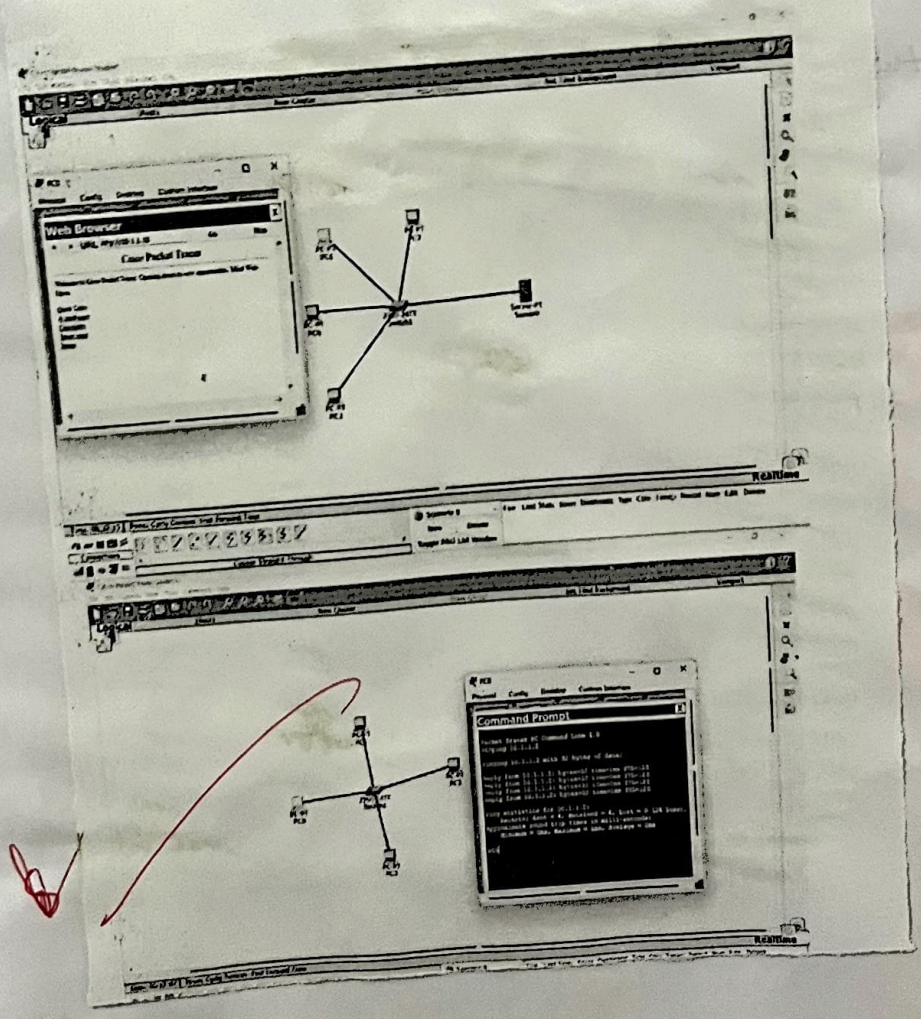
[serial 0/0/0 - 192.168.1.249 <==> Router 1 (serial 0/0/0 - 192.168.1.250/ <==> Router 1.

[serial 0/0/1 - 192.168.1.246 <==> Router 2 (serial 0/0/0 - 192.168.1.245/ <==> Router 2

[Fast Ethernet 0/0 - 20.0.0.1] <==> PC1 / Destination Source - 20.0.0.2)

By default RIP will use the route that has low hops Counts between Source and destination




Web Browser
Command Prompt

28/9/26

Pratical 12.

End - End Communication at Transport Layer.

a) Implement echo clientserver using TCP/UDP Sockets.

b) Implement a chat program using Socket programming.

TCP

```
import Socket
import threading

# ------ Server ----
def start_server():
Server = Socket.Socket(Socket.AF_INET, Socket.Sock-
Stream)
Server, bind ('127.0.0.1', 5000)
Server, listen(1)
Print (" SERVER] listening on 127.0.0.1: 5000")
Conn, addr = Server accept()
Print f" (SERVER) Connected by (addr)")
# ------ CLIENT ------
def start_client():
client = Socket.Socket(Socket.AF_INET, Socket.Sock-
Stream)
client.Connect('127.0.0.1', 5000)
Print (" [client) Connected to server")
while true:
msg = input("Enter message (or quit to exit): ")
client.Send (msg.encode())
if msg.lower() == 'quit';
break.
response = Client.recv(1024), decode()
```

```
print(f"[SERVER RESPONSE] {response}")
client.close()

# ---- MAIN -----
if __name__ == "__main__":
# Run Server in a background thread
server_thread = threading.Thread(target=start_server,
daemon=True)
server_thread.start()
# Run client in main thread.
start_client()
```

UDP

```
import socket
import threading

# ---- Server -----
def udp_server():
s = socket.socket(socket.AF_INET,
socket.SOCK_DGRAM)
s.bind(('127.0.0.1', 12000))
print("[SERVER] Running on 127.0.0.1:12000")
while True:
data, addr = s.recvfrom(1024)
msg = data.decode()
if msg.lower() == 'quit':
print(f"[SERVER] client {addr} ended the
connection.")
break
```

2)

a)

b)

```
Print (f" [SERVER] received from {addr}: {msg}"
S. Sendto (data, addr) # Echo back
S. close ()
Print (" [SERVER] stopped. ")
# ---- CLIENT ------
def udp_client():
c = Socket. Socket (socket. AF_INET,
Socket. Sock_DGRAM)
Server = ('127.0.0.1', 12000)
while True:
msg = input (" Enter message (or 'quit' to stop): ")
c. Sendto (msg. encode(), Server)
if msg. lower () == 'quit':
Print (" [CLIENT] closing Connection ...") break
data, _ = c recvfrom (1024)
Print (" Echo from Server:", data. decode ())
C. close ()
Print (" [client] stopped.")
# ---- MAIN -----
If __name__ == "__main__":
threading. Thread (target = udp_server,
daemon = True). sta
rt () udp_client ()
```

OUTPUT:

12) a

```
[SERVER] Listening on 127.0.0.1:5000[CLIENT] Connected to server

[SERVER] Connected by ('127.0.0.1', 53458)Enter message (or 'quit' to exit):
hello
[SERVER] Received: hello
[SERVER RESPONSE] Server received: hello
Enter message (or 'quit' to exit):
========== RESTART: C:/Users/tkala/Downloads/tcp.py ==========
[SERVER] Listening on 127.0.0.1:5000[CLIENT] Connected to server

[SERVER] Connected by ('127.0.0.1', 60328)Enter message (or 'quit' to exit):
HI
[SERVER] Received: HI
[SERVER RESPONSE] Server received: HI
Enter message (or 'quit' to exit): quit
[SERVER] Connection closed by client.
```

12b)

```
Enter message (or 'quit' to stop): [SERVER] Running on 127.0.0.1:12000hi

[SERVER] Received from ('127.0.0.1', 49670): hi
Echo from Server: hi
Enter message (or 'quit' to stop): hello
[SERVER] Received from ('127.0.0.1', 49670): hello
Echo from Server: hello
Enter message (or 'quit' to stop): how are you
[SERVER] Received from ('127.0.0.1', 49670): how are you
Echo from Server: how are you
Enter message (or 'quit' to stop): quit
[CLIENT] Closing connection...[SERVER] Client ('127.0.0.1', 49670) ended the connection.

[CLIENT] Stopped.[SERVER] Stopped.

>>
```

29/9/26 PRACTICAL 13

13. Implement your own ping program.

```
import os
import platform
import subprocess

def ping(host="google.com", count=4):
    param = "-n" if platform.system().lower()==
    "windows" else "-c"
    command = ["ping", param, str(count), host]
    try:
        output = subprocess.check_output(command, universal_
        newlines=True) print(output)
    except Exception as e,:
        print(f"Ping failed; {e}")
if __name__=="__main__":
    ping("8.8.8.8")
```

Output:

```
Pinging 8.8.8.8 with 32 bytes of data.
Reply from 8.8.8.8: bytes=32 time=6ms TTL=119
Reply from 8.8.8.8: bytes=32 time=9ms TTL=119
Reply from 8.8.8.8: bytes=32 time=5ms TTL=119
Reply from 8.8.8.8: bytes=32 time=9ms TTL=119.
Ping statistics for 8.8.8.8:
Packets: Sent=4, Received=4, lost=0 (0% loss),
Approximate round trip times in milli-seconds:
Minimum=5ms, Maximum=9ms, Average=7ms.
```

PRACTICAL 14

5.10.26

14. Write a code using RAW Sockets to implement packet sniffing.

```
from scapy.all import sniff
from scapy.layers.inet import IP, TCP, UDP, ICMP.

def packet_callback(Packet):
if IP in packet:
ip_layer = Packet[IP]
Protocol = ip_layer.Proto
Src_ip = ip_layer.Src
dst_ip = ip_layer.dst.

# Determine the protocol
protocol name = ""
if protcol = = 1:
 Protocolname = "ICMP"
elif Protocol = = 6;
  Protocol_name = "TCP"
elif protocol = = 17;
   Protocol_name = "UDP"
else;
 Protocol_name = "Unknown Protocol"

# Print Packet details
 Print(f"Protocol: {protocol_name}")
 Print(f"Source IP; {Src_ip}")
 Print(f"Destination IP; [dst_ip]")
 Print("_" * 50)
```

Capture packets on the default network interface.

```
Sniff(iface = "Wi_Fi", prn = Packet_Callback,
filter = "ip", store = 0)
```

output:



Completed & Verified